

Framework para el Desarrollo de Aplicaciones	JMX. La guía perdida. Parte 4. Accediendo de manera remota a un proceso java. <i>Por Isaac Ruiz</i> Publicado en Febrero 2018
Application Express	
Business Intelligence	Table of Contents
Cloud Computing	JMX. La guía perdida. Parte 4. Accediendo de manera remota a un proceso java.
Communications	Alcance.
Rendimiento y Disponibilidad de Base de datos	Requerimientos.
Data Warehousing	Introducción.
.NET	Modificando nuestro ejemplo.
Lenguajes de Programación Dinámicos	Enviando mensajes.
Embedded	Compilación.
Arquitectura Enterprise	Ejecución.
Enterprise Management	Primer escenario. Probando en local.
Grid Computing	Segundo escenario. Remoto sin seguridad.
Identidad y Seguridad	Tercer escenario. Remoto con seguridad. Usuario y password.
Java	Conclusión.
Linux	Enlaces.
Service-Oriented Architecture	Alcance.
SQL & PL/SQL	El presente documento detalla cómo conectarse a un proceso java de manera remota vía JMX, para acceder a su listado de MBeans.
Virtualización	Este documento es la 4ª parte de una serie de artículos sobre JMX, puedes consultar los anteriores aquí: Parte 1. JMX, Java Management Extensions. La guía perdida. Los fundamentos. Parte 2. JMX. La guía perdida. Weblogic y sus MBeans Servers. JMX. La guía perdida. Parte 3: JMX, Weblogic y Multitenant. En el primer artículo de la serie se hace un repaso por las bases de la tecnología JMX, si aún no estás familiarizado, es recomendable leerlo.

Requerimientos.

El presente documento asume que se tiene cierta experiencia construyendo aplicaciones usando el lenguaje de programación java, asume también que el lector tiene experiencia como desarrollador de aplicaciones y conoce en ese nivel un sistema operativo, por ende, sabe ejecutar tareas a nivel CLI.

En este documento, reutilizaremos el código de la parte 1 de esta serie, modificaremos un MBean y mostraremos como realizar una conexión remota a un proceso java para administrarlo con JMX.

Introducción.

En esta 4ª parte comenzaremos a acceder a procesos JMX de manera remota utilizando únicamente lo que nos proporciona el estándar.

Hasta el momento, hemos usado JConsole para acceder a un proceso local y así acceder al servidor de MBeans por default que ofrece dicho proceso, en esta ocasión usaremos JConsole para acceder a un proceso remoto y utilizar así una de las principales bondades de JMX.

Volveremos a utilizar el ejemplo de la primera parte.

Modificando nuestro ejemplo.

En la primera parte de esta serie utilizamos un código de ejemplo para mostrar la construcción de un MBean sencillo; ahora realizaremos una pequeña modificación a dicho ejemplo.

Recordaremos que, el código lo que hace es crear un MBean, lo registra en el servidor de MBeans por default y entra en un ciclo infinito, dentro de ese ciclo espera la entrada de texto por consola, y termina el ciclo cuando se escribe la palabra END. Un ejemplo muy sencillo pero que nos permite ilustrar lo básico de JMX.

Le agregaremos un método a nuestro MBean para poder enviar mensajes utilizando a JConsole como intermediario. Recordemos que una vez conectados a un MBean server, JConsole nos permite inspeccionar los MBeans disponibles en ese servidor, seleccionar uno y acceder a su lista de propiedades, métodos y notificaciones.

Enviando mensajes.

La definición de nuestro MBean ahora es:

```
package com.sps.jmx.mbeans;
/**
 *
 * @author RuGI (S&P Solutions)
 */
public interface ControlMBean {

    public String lastMessage();
}
```

```

    public int    attempts();

    public    void clear();

    public    void sendMessage(String message);
}

```

Hemos agregado sólo un método:

```
public void    sendMessage(String message)
```

Este método nos va permitir enviar mensajes al proceso que queremos acceder.

Nuestra clase para implementar queda:

```

package com.sps.jmx.mbeans;

import java.util.List;
import java.util.Date;
/**
 *
 * @author    RuGI (S&P Solutions)
 */
public class Control implements ControlMBean {

    private    List<String> words;

    public    Control(List<String> words) {
        super();
        this.words = words;
    }

    @Override
    public    String lastMessage() {
        return    (this.words.size() > 0)
            ? this.words.get(this.words.size() - 1)
            : null;
    }

    @Override
    public int    attempts() {
        return    this.words.size();
    }

    @Override
    public    void clear() {
        System.out.println("Que    suerte!. Se ha reiniciado su contador
de intentos. ");
        this.words.clear();
    }

    @Override
    public    void sendMessage(String message) {
        System.out.println(new Date().toString()+":"+message);
    }
}

```

Y, por último, la clase para integrar todo queda así:

```

package com.sps.jmx.juegos;

import java.lang.management.ManagementFactory;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
import javax.management.MBeanServer;
import javax.management.ObjectName;
import com.sps.jmx.mbeans.Control;

/**
 *
 * @author    RuGI (S&P Solutions)
 */
public class Adivina {

    private    List<String> words;
    private    StringBuffer endControl;

    public    Adivina(List<String> words, String endWord) {
        super();
        this.words = words;
        this.endControl = new StringBuffer(endWord);
    }

    public    void addWord(String word) {
        this.words.add(word);
    }

    public int    getNumberWords() {
        return    this.words.size();
    }

    public    List<String> getWords() {

```

```

        return    this.words;
    }

    public static void main(String[] args) throws Exception {
        Adivina adivina = new Adivina(new ArrayList<String>(),    "END");
        MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();
        ObjectName name
        = new
        ObjectName("mx.com.spsolutions.jmxtutorial:type=Control");
        Control mbean = new Control(adivina.getWords());
        mbs.registerMBean(mbean, name);
        Scanner keyboard = new Scanner(System.in);
        String    input;
        do {
            input = keyboard.nextLine();
            System.out.println("Escribiste:" + input);
            adivina.addWord(input);
        } while (!input.equals(adivina.endControl.toString()));
        System.out.println("Adivinaste en [" + adivina.getNumberWords() +
        "]" intentos.");
    } //main
} //class

```

Compilación.

Compilamos estas 3 clases y nuestro ejemplo estará listo para ser ejecutado.

Requerimos compilar:

```

%> javac com/sps/jmx/mbeans/ControlMBean.java
%> javac com/sps/jmx/mbeans/Control.java
%> javac com/sps/jmx/juegos/Adivina.java

```

Ejecución.

Una vez compiladas las clases podemos probar así.

```

%> java com/sps/jmx/juegos/Adivina.java

```

Y la consola debe estar esperando que tecleemos texto seguido de ENTER, el ciclo se romperá cuando escribamos END, seguido también de la tecla ENTER.

Este es un ejemplo de una ejecución:

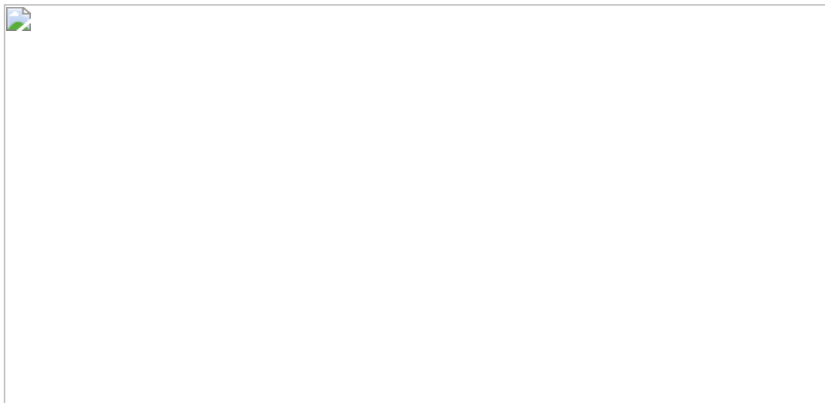


Figura 1. Ejemplo de ejecución de nuestro código.

Ahora, repetiremos esta ejecución con 3 escenarios.

Primer escenario. Probando en local.

Nuestro primer escenario es, el que ya conocemos, dentro de la misma maquina ejecutamos JConsole.

Nuestro programa se ejecuta sobre la JVM y esta tiene un servidor de MBeans por default, es en este servidor donde registramos nuestro MBean, justo en estas líneas:

```

        Adivina adivina = new Adivina(new ArrayList<String>(),    "END");

        MBeanServer mbs =    ManagementFactory.getPlatformMBeanServer();

        ObjectName name
        = new
        ObjectName("mx.com.spsolutions.jmxtutorial:type=Control");

        Control mbean = new    Control(adivina.getWords());

        mbs.registerMBean(mbean, name);

```

Si ejecutamos JConsole en esa misma maquina podemos acceder sin problemas a ese proceso.

Figura 2. Accediendo de manera local a un proceso java via JMX.

Vamos a repetir la ejecución de nuestro ejemplo para poder accederlo de manera local:

```
%>java com/sps/jmx/juegos/Adivina.java
```

Dado que JConsole en su pantalla inicial muestra un listado de los procesos locales, únicamente seleccionamos el que hace referencia a nuestra ejecución.

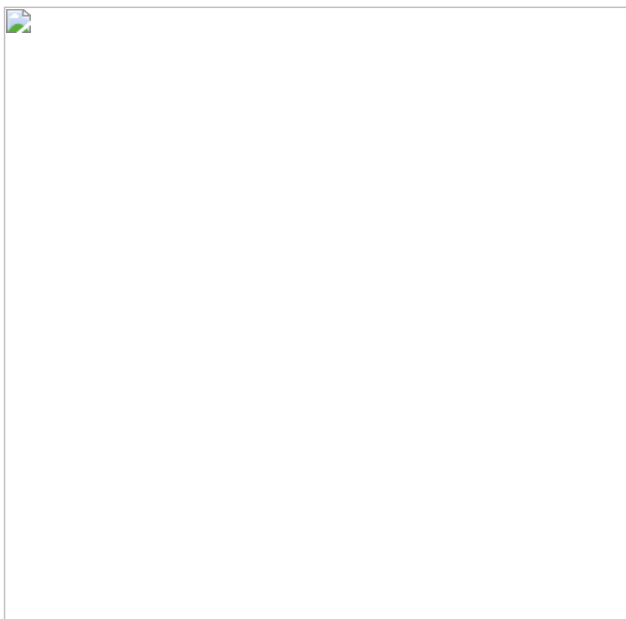


Figura 3. Listado de procesos locales ejecutándose en la JVM.

La advertencia de seguridad respecto a que no estamos usando SSL aparecerá (Insecure connection), la ignoramos y ya podremos ver nuestro MBean.

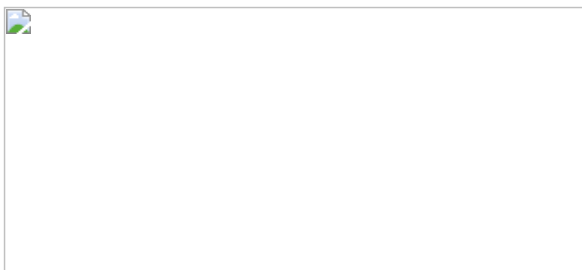


Figura 4. Excepción de seguridad.

JConsole tiene el *tab* de MBeans en la parte derecha de la interface de usuario, seleccionamos [MBeans] y a la izquierda aparece el listado de MBeans, nuestro MBean debe aparecer en el listado.

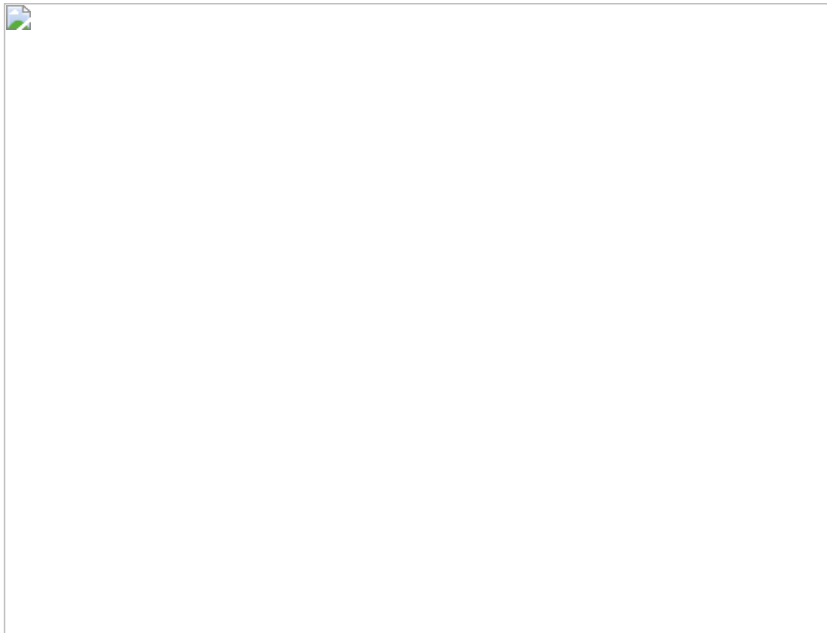


Figura 5. Localizando nuestro MBean.

Si seleccionamos [Operations] podemos ver la lista de operaciones que tiene nuestro MBean. Ya podemos observar la operación sendMessage.

Probaremos la operación, agregando un mensaje (en la imagen "Hola") y haciendo click en el nombre de la operación: [sendMessage].

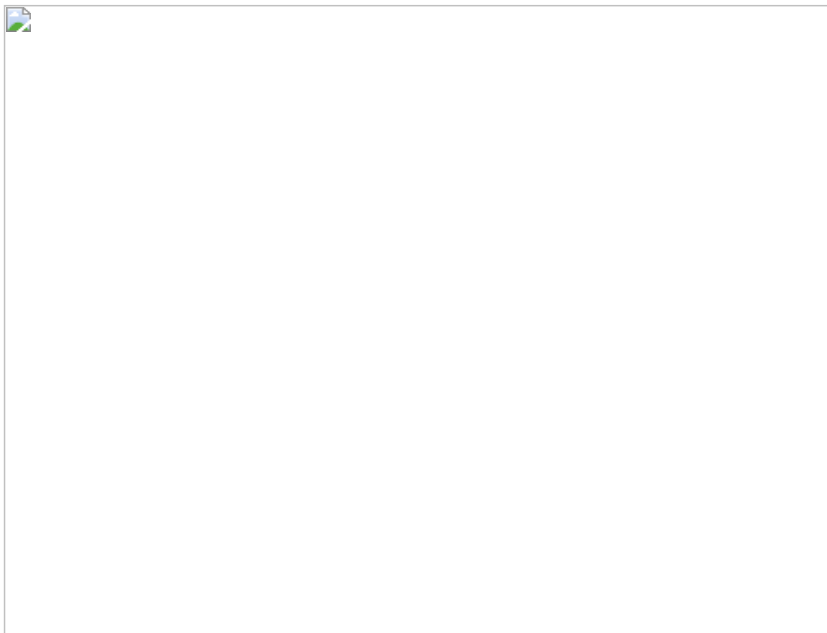


Figura 6. Enviando un mensaje desde JConsole.

Esto hará que la palabra "Hola" se muestre en la ejecución de nuestro proceso java. Veremos un mensaje similar al siguiente:

```
Tue Nov 07 08:54:49 CST 2017:Hola
```

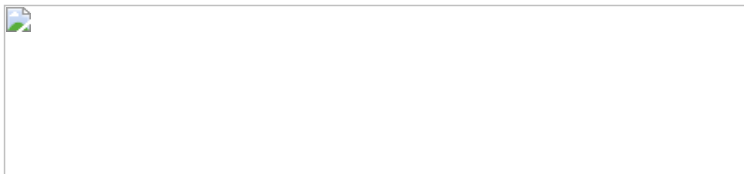


Figura 7. Recibiendo un mensaje desde JConsole.

Con esto hemos probado nuestra nueva operación, y estamos listos para continuar con los otros escenarios un poco más interesantes.

Segundo escenario. Remoto sin seguridad.

Así como podemos acceder a un proceso de manera local, también lo podemos hacer de manera remota, de hecho, es de las principales bondades de JMX.

A continuación, veremos cómo acceder a nuestro ejemplo de manera remota.

Nuestro escenario será ahora el siguiente:

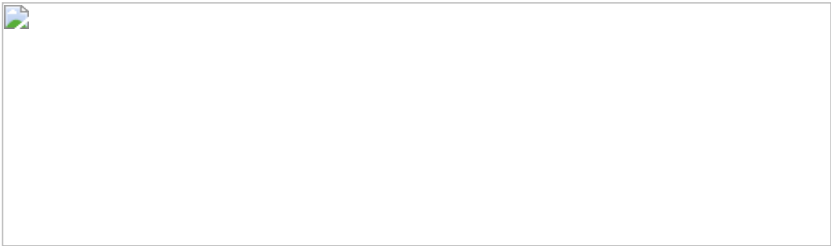


Figura 8. Accediendo de manera remota a un proceso java via JMX. Sin seguridad.

Asumiremos que tenemos dos máquinas que están conectadas a una misma red. En la maquina 2 estará ejecutándose nuestro ejemplo, y desde otra máquina (máquina 1) accederemos vía JMX utilizando JConsole para invocar la operación de nuestro MBean.

Realizaremos primero una conexión sin seguridad, para familiarizarnos con algunos parámetros que debemos conocer y posteriormente le agregaremos un nivel de seguridad al acceso.

Para poder acceder de manera remota, debemos especificar con ciertas propiedades desde línea de comandos (también se puede con código, pero exploraremos primero la versión CLI) que el proceso que vamos a ejecutar va a estar disponible a través de cierto puerto y con la seguridad desactivada.

Existen en particular 4 propiedades que permiten hacer esto, estas propiedades son asignadas vía línea de comandos y el proceso al iniciar los toma y se comporta según lo indicado.

Estas propiedades son:

Propiedad	Descripción
com.sun.management.jmxremote	Hasta la versión 6 esta propiedad era necesaria, a partir de la 6 ya no necesaria. Habilita el acceso remoto a la JVM local Por ejemplo: com.sun.management.jmxremote=true
com.sun.management.jmxremote.port	Indica en que puerto se expone la conexión JMX (vía RMI). Por ejemplo: -Dcom.sun.management.jmxremote.port=8008
com.sun.management.jmxremote.authenticate	Indica si se requiere o no autenticación por password. Por ejemplo: com.sun.management.jmxremote.authenticate=false
com.sun.management.jmxremote.ssl	Habilita o deshabilita la seguridad vía SSL. Por ejemplo: com.sun.management.jmxremote.ssl=false

Estas no son todas las propiedades que podemos utilizar al momento de iniciar un proceso Java y que tienen que ver con JMX, podemos ver más propiedades de configuración en el siguiente enlace:

Monitoring and Management Using JMX Technology. Chapter 2.
<https://docs.oracle.com/javase/8/docs/technotes/guides/management/agent.html>

Dado lo anterior, usaremos estas 4 propiedades con los siguientes valores:

Propiedad	Valor
com.sun.management.jmxremote	No es necesario indicarla si usas una JVM superior a la 6
com.sun.management.jmxremote.port	8008
com.sun.management.jmxremote.authenticate	False

com.sun.management.jmxremote.ssl	False
----------------------------------	-------

La línea de invocación queda entonces (esto en la maquina 2):

```
%>java -Dcom.sun.management.jmxremote.port=8008  
-Dcom.sun.management.jmxremote.authenticate=false  
-Dcom.sun.management.jmxremote.ssl=false  
com.sps.jmx.juegos.Adivina
```

(Con -D se indica que estamos asignando un valor a una propiedad).

Observa que el puerto que estamos indicando es el 8008. Con esto, nuestro proceso está listo para poder ser accedido de manera remota.

Realizaremos lo siguiente, desde la maquina 1 (conectada a la misma red) ejecutaremos JConsole. En la pantalla de JConsole ingresamos la IP de la máquina y el puerto que indicamos anteriormente (8008).

Para este ejercicio asumiremos que la IP de la maquina 2 es: 10.0.1.29, así la dirección remota que utilizaremos será: 10.0.1.29:8008; tal como se muestra a continuación.

Figura 9. Accediendo de manera remota, indicando IP y puerto

Dejamos vacíos los campos: username/password, y damos click en [Connect].

Listo, nos hemos conectado remotamente a nuestro proceso java.



Figura 10. Enviando un mensaje a un proceso remoto.

Podemos ver sin problemas a nuestro MBean y podemos ejecutar la operación [sendMessage] sin problemas. Nuestro primer mensaje de manera remota ha sido enviado :)

Tercer escenario. Remoto con seguridad. Usuario y password.

Ahora agregaremos un nivel de seguridad: solicitar usuario y password para poder conectarnos remotamente; en JMX la seguridad basada en usuario y password requiere que conozcamos dos archivos en particular.

Uno de los archivos define los roles (al mismo tiempo son los *username* para firmarse en JConsole) y el otro los respectivos *passwords* para cada rol.

Estos son:

```
jmxremote.access
Define los roles.
jmxremote.password
Define los passwords de cada rol.
```

Ambos archivos ya existen y están listos para ser utilizados, ya sea que tengamos instalado el JDK o sólo el JRE se encuentran en la carpeta lib/management

Si tienes instalado el JDK:

```
$JDK_HOME/jre/lib/management
Si tienes sólo el JRE:
```

```
$JRE_HOME/lib/management
```

Para este ejemplo, asumimos que tenemos instalado el JDK:

```
$JDK_HOME/jre/lib/management> dir jmx*

jmxremote.access

jmxremote.password.template
```

Abriremos primero el archivo jmxremote.access, este archivo muestra los roles con los que podemos conectarnos vía JMX, si bien es posible poder crear nuestros propios roles, existen dos por *default* que cubren las necesidades más comunes: solo lectura(monitorRole) y lectura-escritura(controlRole).

Figura 11. Archivo jmxremote.access.

A este archivo, jmxremote.access, no le haremos nada, tiene los roles que necesitamos y se encuentra en dónde debe de estar.

Ahora conozcamos el contenido del archivo: `jmxremote.password.template`, como su nombre lo indica, es un *template* a partir del cual podemos crear el que realmente vamos a utilizar.

Copiaremos ese archivo a la carpeta del usuario (o a otra que deseemos) y lo renombraremos por: `jmxremote.password`

Al abrirlo veremos esto al final del archivo estas dos líneas (comentadas por el carácter #):

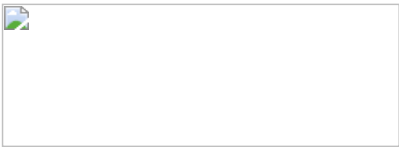


Figura 12. Archivo `jmxremote.password`.

Vamos a *descomentar* el archivo, quitándole los signos #, el archivo queda:

Figura 13. Archivo `jmxremote.password`, sin comentarios.

El archivo lo que indica es que, el usuario `monitorRole` tendrá como password `QED`. Y el usuario `controlRole`, su password será: `R&D`

Sencillo, ¿Certo?

Sólo nos falta algo para utilizar este archivo, por razones de seguridad, el archivo de passwords requiere cierto nivel de permisos; en concreto, sólo el usuario que lo va a utilizar (y nadie más) puede leer y escribir sobre él.

En el caso de que estés probando en una maquina *nix, puedes realizar esto de la siguiente manera:
`chmod 600 jmxremote.password`

En el caso de Windows, es posible realizar esta asignación de permisos de la siguiente manera:

```
cacls jmxremote.password /P Owner:R
```

Si tu versión de Windows no tiene la utilidad `cacls`, revisa el siguiente enlace para conocer el equivalente:
<https://docs.oracle.com/javase/8/docs/technotes/guides/management/security-windows.html>

Para utilizar este archivo, nos apoyaremos en otra propiedad:

Propiedad	Descripción
<code>com.sun.management.jmxremote.password.file</code>	Indica la ubicación del archivo que contiene los passwords a utilizar para los roles definidos en el archivo: <code>jmxremote.access</code> Por ejemplo: <code>com.sun.management.jmxremote.password.file=/export/home/usuario/jmxremote.password</code>

Una vez que el archivo tiene los permisos adecuados, continuamos.

Ahora tenemos algo como lo siguiente:

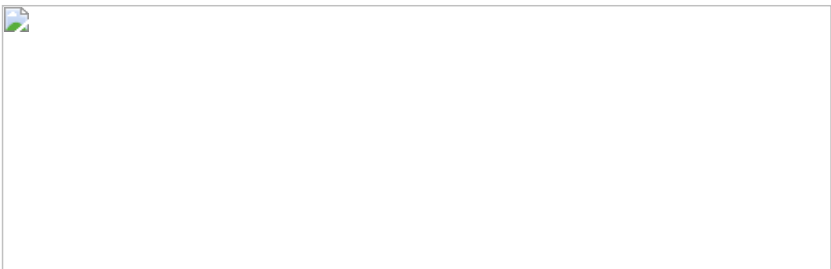


Figura 14. Accediendo de manera remota a un proceso java via JMX, con autenticación sencilla.

Listo, vamos a ejecutar nuevamente nuestro ejemplo en la máquina 2, pero ahora con la siguiente línea:

```
%>java -Dcom.sun.management.jmxremote.port=8008  
-Dcom.sun.management.jmxremote.authenticate=true  
-Dcom.sun.management.jmxremote.ssl=false
```

```
-Dcom.sun.management.jmxremote.password.file=/ruta/al/archivo/jmxremote.password  
com.sps.jmx.juegos.Adivina
```

Asegúrate de que la ruta al archivo de passwords sea válida.

Desde la maquina 1 ejecutamos JConsole, entraremos primero como monitorRole, el password es: QED, la IP sigue siendo la misma y el puerto sigue siendo el mismo: 10.0.1.29:8008



Figura 15. Accediendo de manera remota con usuario monitorRole.

Listo, hemos accedido como monitorRole, el cual recordaremos tiene solo permisos de únicamente lectura.

¿Qué pasa si queremos invocar alguna operación? Intentémoslo.

Figura 16. Excepción generada por falta de privilegios.

Se lanza una excepción indicando que no podemos ejecutar esa operación (de hecho, por nuestros privilegios, no podemos ejecutar ninguna operación).

Cerramos el JConsole y lo volveremos a ejecutar, ahora usaremos el usuario controlRole, el password es: R&D

Misma IP y mismo puerto: 10.0.1.29:8008

Figura 17. Accediendo de manera remota, indicando IP y puerto, usuario y password.

Ahora sí podemos ejecutar las operaciones.

Figura 18. Enviando otro mensaje.

Felicitaciones, tenemos ya asegurado el acceso a nuestro proceso JMX usando una autenticación sencilla basada en usuario y password.

Conclusión.

En esta cuarta entrega sabemos ya como activar el monitoreo remoto de una aplicación java, hemos realizado este monitoreo con JConsole y hemos agregado un nivel de seguridad a la conexión.

En las siguientes entregas veremos cómo utilizar JConsole para monitorear servidores de aplicaciones, como WLS 12c.

Enlaces.

JSR-000003: JavaTM Management Extensions (JMX) Specification.

<https://jcp.org/en/jsr/detail?id=3>

JSR-000003 JavaTM Management Extensions (JMX) (Maintenance Release 4)

<https://jcp.org/aboutJava/communityprocess/mrel/jsr003/index4.html>

JSR 000255 - JMX Specification, version 2.0

<https://jcp.org/en/jsr/detail?id=255>

JSR 160: Java Management Extensions (JMX) Remote API.

<https://jcp.org/en/jsr/detail?id=160>

Java SE Monitoring and Management Guide
<https://docs.oracle.com/javase/8/docs/technotes/guides/management/agent.html>
Repositorio con el código usado en esta serie de artículos.
<https://github.com/rugi/jmx>

Isaac Ruiz Guerra (@rugi), es programador Java yConsultor TI. Especializado en integración de sistemas, fundamentalmente relacionados con el sector financiero. Actualmente forma parte del equipo de S&P; Solutions. Escribe en su blog personal xhubacubi.blogspot.com, pero también participa y pertenece a www.javahispano.org y a www.javamexico.org

Este artículo ha sido revisado por el equipo de productos Oracle y se encuentra en cumplimiento de las normas y prácticas para el uso de los productos Oracle.

 E-mail this page  Printer View

Contáctenos

- Ventas en EE. UU.:
+1.800.633.0738
- Reciba una llamada de Oracle
- Contactos globales
- Directorio de soporte

Acerca de Oracle

- Información de la empresa
- Comunidades
- Carreras

Cloud

- Descripción general de Cloud Solutions
- Software (SaaS)
- Plataforma (PaaS)
- Infraestructura (IaaS)
- Datos (DaaS)
- Prueba gratuita de la nube

Eventos

- Oracle Code
- JavaOne
- Todos los eventos de Oracle

Acciones principales

- Descargue Java
- Descargue Java para desarrolladores
- Pruebe Oracle Cloud
- Suscríbase a los correos electrónicos

Noticias

- Sala de prensa
- Revistas
- Historias de éxito de los clientes
- Blogs

Temas clave

- ERP, EPM (Finanzas)
- HCM (RR. HH. y talento)
- Mercadeo
- CX (Ventas, servicio, comercio)
- Soluciones sectoriales
- Base de datos
- MySQL
- Middleware
- Java
- Sistemas de ingeniería